

NTMS Terraform D-6

27 Questions

1. Why is state locking necessary when using a remote backend?

- 0/0 **A** encrypts the state file at rest and in transit during all Terraform operations to ensure the security of sensitive values
- 0/0 **B** prevents concurrent runs from writing to the same state at the same time to avoid state corruption.
- 0/0 **C** automatically roll back partial changes if an apply fails on any targeted resource to improve cost savings
- 0/0 **D** make plans and applies faster by caching state data on the backend server for Terraform to access

2. You are using a local backend and accidentally delete the terraform.tfstate file for your workspace. What is the most serious consequence?

- 0/0 **A** Terraform will be required to recreate the missing state file by reading values from variables and outputs in the configuration.
- 0/0 **B** Terraform can no longer track the resources it manages, so the next plan or apply might attempt to create duplicate resources.
- 0/0 **C** Terraform will require you to switch to using a remote backend until a new local state file is created manually.
- 0/0 **D** Terraform will need to download the last good state file from the provider API and can then continue managing the resources normally.

3. In Terraform, what does “drift” mean in the context of a workspace’s state?

- 0/0 **A** The backend was migrated, but the state contents remained exactly the same
- 0/0 **B** Provider versions were upgraded but no resources or state values changed at all.
- 0/0 **C** The configuration files were refactored and now use different argument ordering.
- 0/0 **D** Real infrastructure has changed outside Terraform and no longer matches the desired state.

4. Why should users not commit the terraform.tfstate file to version control? (select two)
- 0/0 ☐ A Committing the file to version control prevents Terraform from reading it to properly manage state.
 - 0/0 ☐ B State can include plaintext secrets and detailed resource data and commit history can expose them.
 - 0/0 ☒ C Storing state in Git prevents you from safely updating infrastructure by reverting a resource change.
 - 0/0 ☒ D VCS provides no state locking, so concurrent runs can cause conflicting commits and corrupt state.
5. A resource was changed manually outside of Terraform. You don't want to make any changes yet, but you want to see how the state would be updated to match the current real-world values. Which command should you run?
- 0/0 ☐ A terraform apply -refresh-only
 - 0/0 ☐ B terraform validate
 - 0/0 ☒ C terraform plan -refresh-only
6. You've decided to change your backend configuration from S3 to HCP Terraform. After updating the backend block, you run terraform init. Which flag should you use to reconfigure the backend without copying the existing state?
- 0/0 ☐ A terraform init -migrate-state
 - 0/0 ☐ B terraform init -force-copy
 - 0/0 ☒ C terraform init -reconfigure
7. Which statements accurately describe tradeoffs between local and remote Terraform state? (select two)
- 0/0 ☒ A Remote state centralizes storage with encryption, locking, and access policies for collaboration.
 - 0/0 ☐ B Local state is best for multi-user teams because changes are automatically merged by Terraform.
 - 0/0 ☒ C Local state is simple and works offline, but lacks shared access, locking, and org-level controls.
 - 0/0 ☐ D Remote state removes the need to secure state files since they never contain sensitive values.

8. You're configuring an S3 backend for your Terraform project. You want to keep sensitive values, such as the bucket name and region, out of version control while keeping other backend configuration in your code. Which approach correctly implements partial backend configuration?
- 0/0 ☐ A create a separate [backends.tf](#) file with the sensitive values and add it to .gitignore
- 0/0 ☐ B store all backend configuration in [terraform.tfvars](#) since it supports backend settings and add it to .gitignore
- 0/0 ☐ C create and define your own environment variables TF_BACKEND_BUCKET and TF_BACKEND_REGION to pass the bucket and region values
- 0/0 ☒ D define the backend block with only the type, then pass the bucket and region values using -backend-config flag during terraform init
9. You're moving a project to a remote backend so the state is stored in Amazon S3. How do you correctly configure and initialize the backend?
- 0/0 ☒ A Define the S3 backend in the terraform block using a backend block - run terraform init to migrate your local state.
- 0/0 ☐ B Create the S3 bucket using resource blocks and reference them from the backend using variables - run terraform apply to migrate the backend.
- 0/0 ☐ C Put the backend block inside a module and set the arguments using input variables - run terraform plan which will migrate state.
- 0/0 ☐ D Set TF_INPUT and TF_DATA_DIR environment variables and run terraform init - Terraform will create the backend and move the state automatically.
10. Which of the following Terraform files should be ignored by Git when committing code to a repo?
- 0/0 ☒ A terraform.tfstate
- 0/0 ☐ B [outputs.tf](#)
- 0/0 ☐ C [terraform.tf](#)
- 0/0 ☐ D [.terraform.lock.hcl](#)
11. True or False? After successfully applying a moved block to refactor your resources, you should immediately remove the moved block from your configuration to keep your code clean.
- 0/0 ☐ T True
- 0/0 ☒ F False

12. You have created a brand-new Terraform repo that has no backend block. After successfully running your first terraform apply, where does Terraform store state by default?

- 0/0 ☒ **A** in the current working directory in a file named terraform.tfstate
- 0/0 ☐ **B** in a temporary OS directory in a file name with a randomly created filename
- 0/0 ☐ **C** in the directory \$HOME/.terraform/state/ in a file named terraform.tfstate
- 0/0 ☐ **D** in an S3 bucket found under the module path in a file that matches the project name

13. True or False? All remote backends in Terraform support state locking by default, so you never need to worry about concurrent modifications when using any remote backend.

- 0/0 ☐ **T** True
- 0/0 ☒ **F** False

14. You're comparing local state versus remote state backends for your team's infrastructure. Which statements below correctly describe the key differences or tradeoffs between these approaches? (select three)

- 0/0 ☒ **A** Remote backends typically provide encryption at rest and access controls for state data, while local state security depends entirely on filesystem permissions
- 0/0 ☐ **B** Local state supports more Terraform providers than remote backends because it has direct filesystem access
- 0/0 ☒ **C** Remote backends provide centralized state access for team collaboration, while local state requires manual file sharing between team members
- 0/0 ☒ **D** Local state is simple to set up with no additional infrastructure required, while remote backends require configuration and maintenance of backend services
- 0/0 ☐ **E** Local state automatically provides encryption at rest and in transit, while remote backends store everything in plaintext by default

15. You're using the local backend. In the current workspace, terraform.tfstate was accidentally deleted, but the configuration is unchanged. You run terraform plan.

Which statements are true? (select three)

- 0/0 ☐ **A** Previous terraform output values are still available.
- 0/0 ☒ **B** terraform destroy cannot identify any resources to destroy until state is restored
- 0/0 ☒ **C** Terraform will try to create previously managed resources.
- 0/0 ☐ **D** State locking will prevent the plan from running until the state is restored.
- 0/0 ☒ **E** Data sources may still read remote systems during planning, but they do not fix the resource state.

16. After running terraform apply and provisioning new infrastructure, a team member accidentally deletes the terraform.tfstate file from the remote directory. What happens when you run terraform plan?

- 0/0** ☐ **A** Terraform will show an error and refuse to run until the state file is manually restored
- 0/0** ☐ **B** Terraform will restore from the backup state file ([.terraform.tfstate.backup](#)) and continue working normally
- 0/0** ☒ **C** Terraform will show that it needs to create all resources as new because it has no record of existing infrastructure.
- 0/0** ☐ **D** Terraform will automatically recover the state by querying the provider APIs to rebuild state information

17. True or False? A remote backend configuration is required for using Terraform.

- 0/0** ☐ **T** True
- 0/0** ☒ **F** False

18. Which operations will update your Terraform state? (select three)

- 0/0** ☐ **A** terraform fmt
- 0/0** ☒ **B** terraform destroy
- 0/0** ☐ **C** terraform plan
- 0/0** ☒ **D** terraform import
- 0/0** ☐ **E** terraform graph
- 0/0** ☒ **F** terraform apply

19. You're writing a backend configuration block for S3. Which of the following can be defined directly inside the backend block?

- 0/0** ☐ **A** values like bucket name, key, and region, plus any computed values from a locals block
- 0/0** ☐ **B** values like bucket name, key, and region, as well as any resource outputs using data source lookups
- 0/0** ☒ **C** values like bucket name, key, and region, but not computed values or variables
- 0/0** ☐ **D** alues like bucket name, key, and region, plus any Terraform variable using var.variable_name syntax

20. Two DevOps engineers have run terraform apply at the same time against the same workspace that uses a remote backend. What are the correct outcomes? (select two)

- 0/0 ☒ **A** If the backend does not support locking, the state file could become corrupted.
- 0/0 ☒ **B** If the backend supports locking, the first terraform apply will lock the file for changes, preventing the second user from running the apply.
- 0/0 ☐ **C** All remote backends enforce locking, so only one apply can ever run at a time.
- 0/0 ☐ **D** Since Terraform automatically merges concurrent state updates, both will succeed without conflict.

21. You open the terraform.tfstate file in a text editor to inspect it. What format is the file, and what type of information does it contain?

- 0/0 ☒ **A** SON format containing resource metadata, attributes, dependencies, and potentially sensitive values in plaintext
- 0/0 ☐ **B** Encrypted binary format that requires the terraform show command to decrypt and view the contents
- 0/0 ☐ **C** YAML format containing detailed information on resource IDs and types but sensitive data is hashed
- 0/0 ☐ **D** HCL format that mirrors your configuration file structure with additional computed attributes

22. All your infrastructure is managed by Terraform, but engineers occasionally make small updates directly on the target platform. Which Terraform command can identify changes to the actual infrastructure and reflect them in the Terraform state?

- 0/0 ☒ **A** terraform apply -refresh-only
- 0/0 ☐ **B** terraform validate
- 0/0 ☐ **C** terraform graph
- 0/0 ☐ **D** terraform show

23. You are configuring remote state and must keep backend credentials out of both the .terraform directory and any saved plan files. Which method will meet this requirement?

- 0/0 ☐ **A** add your secrets in [terraform.tfvars](#) and pass -var-file when running plan/apply
- 0/0 ☒ **B** supply the credentials using environment variables on the machine executing plan/apply
- 0/0 ☐ **C** add secrets in the provider block and configure a .gitignore to exclude that file
- 0/0 ☐ **D** pass secrets with -backend-[config=secrets.hcl](#)

24. You run a standard terraform plan (without any flags) on your configuration. During the planning phase, what does Terraform do with the current state?

- 0/0 ☐ A Terraform skips state refresh entirely unless you explicitly use the -refresh=true flag
- 0/0 ☒ B Terraform automatically queries the real infrastructure to refresh state data before generating the plan, but doesn't save the refreshed state to the file
- 0/0 ☐ C Terraform queries real infrastructure and writes the refreshed state to terraform.tfstate before showing the plan
- 0/0 ☐ D Terraform only reads from the existing state file without needing to query the real infrastructure

25. You have recently run terraform apply successfully using the local backend. You notice Terraform has created a new file called [terraform.tfstate.backup](#) in your working directory. What is the purpose of this file?

- 0/0 ☒ A It contains the previous state before the most recent apply, allowing manual recovery if the current state becomes corrupted
- 0/0 ☐ B It's automatically used by Terraform to roll back changes if an apply operation fails
- 0/0 ☐ C It's an encrypted copy of terraform.tfstate used for disaster recovery that requires a decryption key
- 0/0 ☐ D It's a compressed archive of all historical state versions created during the project lifecycle

26. Assuming default settings, which statement best describes how the local backend is used?

- 0/0 ☐ A The local backend requires an S3 bucket or similar cloud-based storage for state locking.
- 0/0 ☐ B The local backend enables remote collaboration out of the box.
- 0/0 ☐ C Terraform automatically encrypts state at rest when using the local backend.
- 0/0 ☒ D Terraform writes state to a file named terraform.tfstate in the current working directory.

- 27.** You have decided to move your state file to an Amazon S3 remote backend. You configure Terraform as shown below. What command should be run in order to complete the state migration while copying the existing state to the new backend?

```
terraform {  
  backend "s3" {  
    bucket = "tf-state-bucket"  
    key    = "terraform/krausen/"  
    region = "ap-south-1"  
  }  
}
```

- 0/0 **A** terraform plan -refresh=false
- 0/0 **B** terraform apply -refresh-only
- 0/0 **C** terraform state show
- 0/0 **D** terraform init -migrate-state